ACADEMIC CAPSTONE PROJECT & PRODUCTION FULL-STACK ARCHITECTURE

Campus Canteen Pre-Order System

A Smart Way to Dine on Campus

An enterprise-grade, web-based dining automation platform engineered to eliminate canteen queues, automate kitchen order dispatching, prevent counter congestion, and provide real-time order tracking with role-based access control.

BACKEND TECHNOLOGY Java 21 LTS & Spring Boot 3.3.4	**FRONTEND TECHNOLOGY** React 18 + Vite (SPA)
DATABASE ARCHITECTURE MySQL 8.x (Relational & Normalized)	**SECURITY & AUTH** BCrypt Hashing & Stateless JWT (RBAC)
PROJECT REPOSITORY github.com/dollibhanu2007/Campus_Canteen	**CLOUD DEPLOYMENT** Render Docker & Static Blueprint

Generated for Technical Review, Viva Voce & Academic Project Presentation

1. Problem Statement & Project Objectives

During peak break hours in universities and colleges, campus canteens face severe congestion. Students spend 15–25 minutes waiting in lines to order and pay, often leading to delayed classes or missed meals. Simultaneously, canteen kitchen staff struggle to predict food demand in advance, leading to inventory shortages or substantial food wastage.

> **Key Innovation:** The Campus Canteen Pre-Order System transforms on-premise dining into a scheduled pickup model. Students pre-order and reserve specific pickup slots prior to their break. When break starts, meals are already prepared and packed for instantaneous token-verified counter handover.

Zero Queue Waiting
Students arrive at the scheduled pickup window, verify their order token, and grab meals without queuing.

Live Kitchen Stream
Kitchen cooks view incoming meals chronologically by time slot and prepare orders in batches.

Audited Financials
Admins monitor real-time gross revenue, net sales excluding cancellations, and item popularity.

2. System Architecture & Order Lifecycle Engine

The system strictly enforces an irreversible, sequential state machine ensuring food is never wasted or cancelled once cooking begins.

Browse Menu & Cart ➔ **PLACED** (Initial Token Issued) ➔ **PREPARING** (Cooked in Kitchen) ➔ **READY** (Packed at Counter) ➔ **COLLECTED** (Handover Complete)

Order Status	Actor Responsible	Business Rules & Lifecycle Constraints
PLACED	Customer	Order created with server-recalculated line totals. Customer cancellation is permitted ONLY at this stage.
PREPARING	Canteen Staff	Chef marks meal as cooking. Customer cancellation is strictly locked and forbidden past this point.
READY	Canteen Staff	Food is packed and waiting at pickup counter. Customer tracker highlights pickup token in green.
COLLECTED	Staff & Customer	Meal handover verified against counter token. Order finalized and logged in completed revenue.
CANCELLED	Customer / Staff	Allowed only in PLACED stage. Reason recorded in DB and excluded from net sales analytics.

3. Role-Based Access Control (RBAC) & Features

1. Student / Staff Customer

- Personalized dashboard with active meal trackers
- Dynamic search and dietary category filters (Veg/Non-Veg)
- Persistent shopping cart with live subtotal
- Scheduled pickup time slot reservation
- Real-time 5-second status polling
- Cancellation permitted strictly when PLACED
- Order itemization and profile settings

2. Canteen Kitchen Staff

- Operational queue board with audio/visual cues
- One-click status updates (Start Preparing, Mark Ready, Handover)
- Real-time inventory stock toggle (In Stock / Sold Out)
- Customer contact details for fulfillment
- Historical kitchen order metrics

3. Campus Administrator

- Platform KPI metrics (Net revenue, total meals, users)
- Date-range financial sales reports
- Full CRUD for Menu items, Categories & Pickup Slots
- User account activation, deactivation & role reassignment
- Administrative audit activity logs

4. Relational Database Schema (MySQL 8)

The schema is normalized to 3NF with foreign key constraints, cascading deletes where appropriate, and price snapshotting.

Table Name	Primary Keys & Relationships	Core Attributes & Purpose
`users`	`id` (PK)	Full name, email (unique), password_hash (BCrypt), role (ROLE_CUSTOMER, ROLE_STAFF, ROLE_ADMIN), phone, active status.
`menu_categories`	`id` (PK)	Category name (Breakfast, Meals, Quick Bites, Beverages), description, display order.
`menu_items`	`id` (PK), FK: category_id	Item name, description, price (₹), available (boolean), is_veg, preparation time, image URL.
`pickup_slots`	`id` (PK)	Start time, end time, max capacity limit per slot, active status.
`orders`	`id` (PK), FK: user_id, pickup_slot_id	Unique order token (e.g. CC-261011-2781), status enum, subtotal, total amount, pickup date, instructions.
`order_items`	`id` (PK), FK: order_id, menu_item_id	**Item price snapshot:** Captures unit price and line totals at purchase time so future menu edits do not corrupt historical financials.
`order_status_history`	`id` (PK), FK: order_id, changed_by	Audit log recording exact transition timestamps, previous status, new status, and kitchen chef notes.
`admin_activity_logs`	`id` (PK), FK: actor_user_id	Tracks administrative modifications including menu pricing changes, user status toggles, and slot capacity alterations.

5. Security & Validation Architecture

Stateless JWT Authentication

Users receive an HMAC-SHA256 signed JWT upon login with a 24-hour expiration window. The token is parsed per-request by Spring Security's `JwtAuthenticationFilter` to populate the SecurityContext.

Server-Side Price Verification

Client-submitted prices and totals are completely ignored. When an order is submitted, Spring Boot fetches current prices from MySQL, recalculates line totals, checks item availability, and validates slot capacity.

6. Execution Instructions & Pre-Seeded Accounts

Starting Backend (Spring Boot)

Run in `backend/` directory:

Starting Frontend (React Vite)

Run in `frontend/` directory:

```
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=standalone"
```

Runs on port 8080 with auto-seeded demo data.

```
npm install && npm run dev
```

Runs on port 5173 with automatic backend proxying.

Role	Login Email	Default Password	Permissions & Capabilities
Student Customer	`student@campus.edu`	`Student@123`	Browse menu, cart checkout, slot picker, order tracking
Kitchen Staff	`staff@campus.edu`	`Staff@123`	Kitchen queue, lifecycle updates, stock availability toggles
Campus Administrator	`admin@campus.edu`	`Admin@123`	System KPIs, revenue reports, full menu & slot CRUD, user control

7. Verification & Acceptance Testing Summary

> **Verification Result:** All 9 backend JUnit 5 / MockMvc tests passed (0 failures). The frontend Vite production bundle compiled 1,660 modules with 0 errors. All 12 end-to-end integration acceptance scenarios (registration, auth, catalog, cart, order token, tracking, kitchen transitions, and RBAC rejection) achieved 100% test success.